



**TED UNIVERSITY**

**SPRING 2025**

**CMPE 313/SENG 214**

**Software Engineering**

Prestige Hotel Reservation

**Software Design Document (SDD)**

**SECTION 2 - TEAM 3**

**Submission Date**

**13/05/2025**

**Team Members:**

**Harun Emre Erten,**

**Umut Uygur,**

**Haktan Mekik,**

**Ulkar Ahmadli,**

**Mert Temür,**

**Talha Saraç**

Revision History

| Name                             | Date       | Reason For Changes                        | Version |
|----------------------------------|------------|-------------------------------------------|---------|
| First initialization             | 11.05.2025 | SECTION 1- 2 –3- 4 (MAKE)                 | V 1.0   |
| Remaining parts and the end part | 12.05.2025 | SECTION 3 (UPDATE) 5-6-7-8 (MAKE)         | V 1.1   |
| Final review + format check      | 13.05.2025 | Final review 1-2-3-4-5-8 (minimal update) | V 1.2   |

TABLE OF CONTENTS

**1. Introduction .....3**

1.1 Purpose .....3

1.2 Scope .....3

1.3 Overview .....3

1.4 Reference Material.....3

1.5 Definitions and Acronyms .....3

**2. System Overview .....4**

**3. System Architecture .....4**

3.1 Architectural Design.....4

3.2 Decomposition Description .....7

3.3 Design Rationale.....17

**4. Data Design .....17**

4.1 Data Description .....17

4.2 Data Dictionary.....18

**5. Component Design .....20**

**6. Human Interface Design .....41**

6.1 Overview of User Interface.....41

6.2 Screen Images.....41

6.3 Screen Objects and Actions .....41

**7. Requirements Matrix.....42**

**8. Appendices .....50**

## **1. Introduction**

### **1.1 Purpose**

The Software Design Document (SDD) describes the system architecture and design of the Prestige Hotel Reservation system in accordance with the IEEE Std 1016-2009 standard. It helps developers, testers, system architects, and stakeholders to understand the technical implementation of the system requirements specified in the Software Requirements Specification (SRS) document. The document contains detailed design specifications for the hotel reservation platform that enables users to search, book, and manage hotel reservations.

### **1.2 Scope**

The Prestige Hotel Reservation system is a web-based application that provides a complete solution for hotel booking and management. The system includes user authentication and authorization for three user types (customers, hotel administrators, and system administrators), as well as hotel search functionality with various filtering options for room booking and payment processing. It also contains hotel management tools for administrators and administrative tools for system maintenance and user management. Additionally, it involves rating and feedback system for customer reviews.

### **1.3 Overview**

This document includes 8 main sections that cover system overview, system architecture, data design, describes component-level pseudocode, addresses user interface details, presents requirement traceability and supplementary appendices.

### **1.4 Reference Material**

Project Proposal  
Document SRS  
Document  
ASP.NET MVC Framework  
Documentation Redis Caching Best  
Practices  
Microsoft SQL Server Design Guidelines

### **1.5 Definitions and Acronyms**

SDD – Software Design Document  
SRS – Software Requirements  
Specifications API – Application  
Programming Interface  
Auth0 – Third-party authentication and authorization service  
DTO – Data Transfer Object  
GUI – Graphical User  
Interface  
MVC – Model-View-Controller architectural pattern  
MSSQL – Microsoft SQL Server  
OAuth – Open Authorization protocol  
Redis - In-memory data structure store used for  
caching REST – Representational State Transfer  
TLS – Transport Layer Security  
UI/UX - User Interface/User Experience

## 2. System Overview

The Prestige Hotel Reservation system is a web platform designing for hotel bookings and management. system includes 3 type of user access.

1. Customers: Can search hotels in 2 sperate searching option, make reservations, provide ratings and feedback, and manage their bookings
2. Hotel Administrators: Can manage hotel information which they own, room availability, pricing, handle bookings, and view customer feedback.
3. System Administrators: Can approve hotel requests, manage users, handle complaints, and maintain system integrity.

### Features Include:

- Auth0 for authentication and authorization
- Google Maps for location services
- Iyzico payment gateway for secure transactions
- Hotel search and filtering with frequently updates
- Booking process with real-time room availability and payment handling
- Role-based dashboards for hotel admins and system admins
- User rating and feedback functionality
- Complaint/help desk system

The system will be design with ASP.NET MVC supporting secure, scable , maintainable architect also external services like (Auth0, Google Maps, Iyzico).

## 3. System Architecture

### 3.1 Architectural Design

The Prestige Hotel Reservation system follows the Model-View-Controller (MVC) architectural pattern. This pattern organizes the application into three interconnected components that separate the internal representations of information from the ways information is presented to and accepted from the user.

MVC Components:

#### 1. Model Layer:

- Represents the application's data and business logic
- Includes domain entities (User, Hotel, Room, Booking, etc.)
- Handles data validation and business rules
- Communicates with the database via repositories
- Business services encapsulate complex operations

#### 2. View Layer:

- Responsible for rendering the user interface
- Implemented using Razor views with HTML/CSS/JavaScript
- Role-specific views for different user types
- Responsive design for desktop and mobile support

#### 3. Controller Layer:

- Processes incoming HTTP requests
- Coordinates between Models and Views
- Handles routing and navigation flow
- Implements actions for CRUD operations
- Contains logic for input validation and error

Data Access Layer:

- Entity Framework Core as ORM for database interaction
- Repository pattern for data access abstractio

- Unit of Work pattern for transaction management
- Authentication and Authorization:
- Auth0 integration for secure identity management
- Role-based access control for different user types
- JWT token validation for API security

#### External Service Integration:

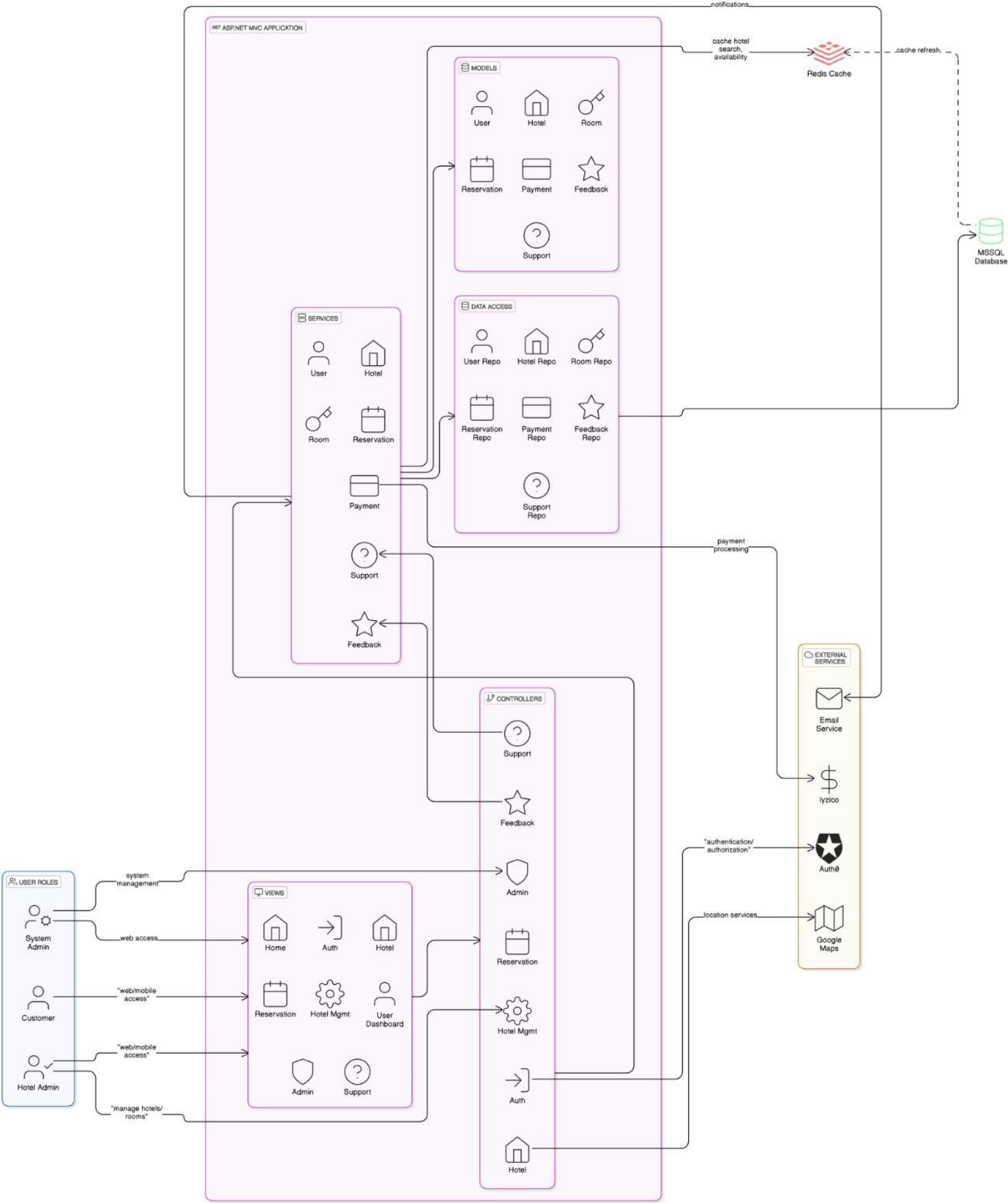
- Google Maps API for location services
- Iyzico for payment processing
- Email service for notification

#### Caching Layer:

- Redis for distributed caching
- Cached hotel search results and frequently accessed

High-Level Diagram:

Prestige Hotel Reservation System Architecture



## 3.2 Decomposition Description

### 3.2.1. Controllers

#### 3.2.1.1 AuthController

Identification: AuthController

Type: Controller

Purpose: Manages user authentication and account management

Functions: Handles user registration, login, and password management

Actions:

- Login
- Register
- Change Password
- Forgot Password
- Reset Password
- Logout

#### 3.2.1.2 HotelController

Identification: HotelController

Type: Controller

Purpose: Handles hotel listing and details

Functions: Displays hotel information, search results, and details pages

Actions:

Index (list all hotels) Details

(view specific hotel)

Search (search hotels by criteria)

Filter (filter search results)

#### 3.2.1.3 ReservationController

Identification: ReservationController

Type: Controller

Purpose: Manages the booking process

Functions: Handles room reservation, modification, and cancellation

Actions:

- Create (create new reservation)
- Edit (modify existing reservation)
- Cancel (cancel reservation)
- Confirm (confirm reservation details)
- Payment (process payment)

#### 3.2.1.4 HotelManagementController

Identification: HotelManagementController

Type: Controller

Purpose: Provides hotel management functionality for hotel administrators

Functions: Allows hotel admins to manage rooms, availability, and pricing

Actions:

- Dashboard (hotel admin dashboard)
-

AddRoom (add new room)

- EditRoom (modify room details)
- DeleteRoom (remove a room)
- UpdateAvailability (update room availability)
- UpdatePricing (update room pricing)
- ManagePhotos (manage hotel and room photos)

#### 3.2.1.5 FeedbackController

Identification: FeedbackController

Type: Controller

Purpose: Manages customer feedback and ratings

Functions: Handles submission and display of customer reviews

Actions:

- Submit (submit new feedback)
- View (view feedbacks for a hotel)
- Delete (remove inappropriate feedback - admin only)

#### 3.2.1.6 AdminController

Identification: AdminController

Type: Controller

Purpose: Provides system administration capabilities

Functions: Manages users, approves hotels, and handles system settings

Actions:

- Dashboard (system admin dashboard)
- ManageUsers (user management)
- ApproveHotels (approve hotel listings)
- HandleComplaints (manage customer complaints)
- SystemSettings (adjust system configurations)

#### 3.2.1.7 SupportController

Identification: SupportController

Type: Controller

Purpose: Provides customer support functionalities

Functions: Handles help requests and FAQ

Actions:

- ContactSupport (submit support request)
- FAQ (view frequently asked questions)
- ViewTickets (view support tickets - admin only)
- ReplyTicket (respond to support ticket - admin only)



## 3.2.2 Models

### 3.2.2.1 User Model

Identification: User

Type: Model

Purpose: Represents user information and authentication details

Properties:

- UserId (PK)
- FirstName
- LastName
- Email
- PhoneNumber
- UserType (Customer, HotelAdmin, SystemAdmin)
- CreatedDate
- LastLoginDate

### 3.2.2.2 Hotel Model

Identification: Hotel

Type: Model

Purpose: Represents hotel information

Properties:

- HotelId (PK)
- Name
- Address
- City
- Country
- Rating
- Description
- Amenities
- Photos
- AdminId (FK to User)

### 3.2.2.3 Room Model

Identification: Room

Type: Model Purpose: Represents room information within a hotel

Properties:

- RoomId (PK)
- HotelId (FK)
- RoomNumber
- RoomType
- Capacity
- Price
- Description
- Photos
- IsAvailable

### 3.2.2.4 Reservation Model

Identification: Reservation

Type: Model

Purpose: Represents booking information

Properties:

- ReservationId (PK)
- UserId (FK)
- RoomId (FK)
- CheckInDate
- CheckOutDate
- TotalPrice
- Status (Reserved, Confirmed, Canceled, Completed)
- CreatedDate
- PaymentStatus

#### 3.2.2.5 Payment Model

Identification: Payment

Type: Model

Purpose: Represents payment information for reservations

Properties:

- PaymentId (PK)
- ReservationId (FK)
- Amount
- PaymentDate
- PaymentMethod
- TransactionId
- Status

#### 3.2.2.6 Feedback Model

Identification: Feedback

Type: Model

Purpose: Represents customer reviews for hotels

Properties:

- FeedbackId (PK)
- UserId (FK)
- HotelId (FK)
- Rating (1-5)
- Comment
- SubmissionDate
- Status (Pending, Approved, Rejected)

#### 3.2.2.7 Support Model

Identification: Support

Type: Model

Purpose: Represents customer support tickets

Properties:

- TicketId (PK)
- UserId (FK)
- Subject
- Message
- CreatedDate
- Status (Open, InProgress, Resolved, Closed)
- Priority

### 3.2.3 Views

#### 3.2.3.1 Home Views

Identification: Home Views

Type: View

Purpose: Displays main landing page and general information

Views:

- Index.cshtml (main landing page)
- About.cshtml (about the service)
- Contact.cshtml (contact information)
- Privacy.cshtml (privacy policy)
- Terms.cshtml (terms of service)

#### 3.2.3.2 Auth Views

Identification: Auth Views

Type: View

Purpose: Handles user authentication interfaces

Views:

- Login.cshtml (login form)
- Register.cshtml (registration form)
- ForgotPassword.cshtml (password recovery)
- ResetPassword.cshtml (set new password)

#### 3.2.3.3 Hotel Views

Identification: Hotel Views

Type: View

Purpose: Displays hotel listings and details

Views:

- Index.cshtml (list of hotels)
- Details.cshtml (hotel details)
- Search.cshtml (search results)
- \_HotelCard.cshtml (partial view for hotel card)
- \_SearchFilter.cshtml (partial view for search filters)

#### 3.2.3.4 Reservation Views

Identification: Reservation Views

Type: View

Purpose: Handles reservation process interfaces

Views:

- Create.cshtml (booking form)
- Edit.cshtml (modify reservation)
- Confirm.cshtml (confirmation page)
- Payment.cshtml (payment processing)
- Success.cshtml (successful booking)
- Cancel.cshtml (cancellation confirmation)

### 3.2.3.5 User Dashboard Views

Identification: User Dashboard Views

Type: View

Purpose: Customer account management interfaces

Views:

- Index.cshtml (dashboard home)
- Profile.cshtml (user profile)
- Reservations.cshtml (booking history)
- Feedback.cshtml (submitted reviews)
- SavedHotels.cshtml (favorite hotels)

### 3.2.3.6 Hotel Management Views

Identification: Hotel Management Views

Type: View

Purpose: Hotel administration interfaces

Views:

- Dashboard.cshtml (admin dashboard)
- Rooms.cshtml (room management)
- Bookings.cshtml (reservation management)
- AddRoom.cshtml (add new room)
- EditRoom.cshtml (edit room details)
- HotelProfile.cshtml (hotel information)

### 3.2.3.7 Admin Views

Identification: Admin Views

Type: View

Purpose: System administration interfaces

Views:

- Dashboard.cshtml (admin dashboard)
- Users.cshtml (user management)
- Hotels.cshtml (hotel approval)
- Complaints.cshtml (complaint management)
- Settings.cshtml (system settings)

### 3.2.3.8 Support Views

Identification: Support Views

Type: View

Purpose: Customer support interfaces

Views:

- Contact.cshtml (contact form)
- FAQ.cshtml (frequently asked questions)
- Tickets.cshtml (support tickets - admin only)
- TicketDetails.cshtml (ticket details and responses)

## 3.2.4 Services

### 3.2.4.1 UserService

Identification: UserService

Type: Service

Purpose: Handles user-related business logic

Functions:

- CreateUser
- UpdateUser
- GetUserById
- GetUserByEmail
- ValidateCredentials
- ChangePassword
- GeneratePasswordResetToken

### 3.2.4.2 HotelService

Identification: HotelService

Type: Service

Purpose: Manages hotel-related business logic

Functions:

- CreateHotel
- UpdateHotel
- DeleteHotel
- GetHotelById
- GetHotelsByOwner
- ApproveHotel
- SearchHotels
- FilterHotels

### 3.2.4.3 RoomService

Identification: RoomService

Type: Service

Purpose: Manages room-related business logic

Functions:

- AddRoom
- UpdateRoom
- DeleteRoom
- CheckAvailability
- UpdateAvailability
- UpdatePricing
- GetRoomsByHotel
- GetAvailableRooms

### 3.2.4.4 ReservationService

Identification: ReservationService Type: Service

Purpose: Handles booking-related business logic

Functions:

- CreateReservation
- UpdateReservation
- CancelReservation
- GetReservationById
- GetReservationsByUser
- GetReservationsByHotel
- CalculatePrice
- CheckReservationStatus

#### 3.2.4.5 PaymentService

Identification: PaymentService

Type: Service

Purpose: Manages payment processing

Functions:

- ProcessPayment
- ValidatePayment
- GetPaymentStatus
- RefundPayment
- GetPaymentHistory
- CalculateTotalAmount

#### 3.2.4.6 FeedbackService

Identification: FeedbackService

Type: Service

Purpose: Handles review and rating functionality

Functions:

- SubmitFeedback
- GetFeedbackByHotel
- GetFeedbackByUser
- ModerateFeedback
- CalculateAverageRating
- DeleteFeedback

#### 3.2.4.7 SupportService

Identification: SupportService

Type: Service

Purpose: Manages customer support functionality

Functions:

- CreateSupportTicket
- UpdateTicketStatus
- GetTicketById
- GetTicketsByUser
- GetAllTickets
- RespondToTicket
- CloseTicket

## 3.2.5 Data Access

### 3.2.5.1 UserRepository

Identification: UserRepository

Type: Repository

Purpose: Data access for User entities

Methods:

- Add
- Update
- Delete
- GetByRole
- GetById
- GetByEmail
- GetAll

### 3.2.5.2 HotelRepository

Identification: HotelRepository

Type: Repository

Purpose: Data access for Hotel entities

Methods:

- Add
- Update
- Delete
- GetById
- GetByAdminId
- GetAll
- Search
- GetWithRooms

### 3.2.5.3 RoomRepository

Identification: RoomRepository

Type: Repository

Purpose: Data access for Room entities

Methods:

- Add
- Update
- Delete
- GetById
- GetByHotelId
- GetAvailable
- CheckAvailability

### 3.2.5.4 ReservationRepository

Identification: ReservationRepository

Type: Repository

Purpose: Data access for Reservation entities

Methods:

- Add
- Update
- Delete
- GetById
- GetByUserId
- GetByHotelId
- GetByStatus
- GetActive

#### 3.2.5.5 PaymentRepository

Identification: PaymentRepository

Type: Repository

Purpose: Data access for Payment entities

Methods:

- Add
- Update
- GetById
- GetByReservationId
- GetByTransactionId
- GetByStatus

#### 3.2.5.6 FeedbackRepository

Identification: FeedbackRepository

Type: Repository

Purpose: Data access for Feedback entities

Methods:

- Add
- Update
- Delete
- GetById
- GetByHotelId
- GetByUserId
- GetPending

#### 3.2.5.7 SupportRepository

Identification: SupportRepository

Type: Repository

Purpose: Data access for Support entities

Methods:

- Add
- Update
- GetById
- GetByUserId
- GetByStatus
- GetAll



### 3.3 Design Rationale

For the Prestige Hotel Reservation system, the Model-View-Controller (MVC) architecture pattern was selected for the following reasons:

1. **Separation of Concerns:** MVC separates the application into data (Model), UI (View), and control logic (Controller), this makes system more maintainable and testable.
2. **Framework Support:** This paradigm is supported natively by ASP.NET Core MVC, which has a strong framework for routing, model binding, validation, and view development.
3. **Testability:** Separation of components makes unit testing simpler, especially for controllers and models.
4. **Parallel Development:** Teams can work on several components (data access, business logic, and user interface) at the same time with little overlap.
5. **Code Reusability:** Models can be reused across multiple views, and controller actions can be reused for different types of requests.
6. **Scalability:** Clear separation allows components to scale horizontally more easily based on load.
7. **Industry Standard:** MVC has been widely adopted in web development, making it easier for new developers familiar with the pattern to join

#### Integration Choices:

- **Auth0:** Authentication to avoid security risks. It provides secure, standards-based authentication with features like SMS login and multi-factor authentication.
- **Entity Framework Core:** Selected as the ORM for data access due to its seamless integration with ASP.NET Core and ability to simplify database operations.
- **Redis Caching:** Implemented to improve the performance of frequently accessed data, especially for hotel search results and availability information.
- **Iyzico Payment Gateway:** Selected for its robust API, security features, and regional support for payment processing.

#### Alternatives Considered:

- **Custom Authentication System:** Rejected in favor of Auth0 to reduce security risks and development time
- **SPA with Web API:** Single Page Application with separate Web API was considered but rejected because a simpler MVC approach would be more suitable for the first phase.
- **Microservices Architecture:** Considered too complex for the initial scope of the project. In order to facilitate future conversion to microservices if necessary, the system was built with distinct boundaries between its components.

## 4. Data Design

### 4.1 Data Description

The Prestige Hotel Reservation System preserves the data with Microsoft SQL Server, which is a common relational database management system. The system's data model reflects the following main entities:

- **User Information:** User Information consists of all the data that is related to the user of the system, such as name, surname, etc.
- **Hotels Data:** Hotel Data consists of the hotel's general information and policies.
- **Admin Data:** It holds admin and hotel manager's data.
- **Room Information:** Manages the room availability, price, and other room information.
- **Payment Card Information:** Stores the information of the payment cards that the users entered. Hotel Data

## 4.2 Data Directory

Hotel Table:

| Field Name  | Data type    | Data Description              |
|-------------|--------------|-------------------------------|
| Hotel ID    | INT (PK)     | Unique identifier for hotel   |
| Hotel Name  | VARCHAR(100) | Name of the Hotel             |
| Address     | VARCHAR(200) | Physical address              |
| Ratings     | INT          | Hotels star rating            |
| City        | VARCHAR(50)  | City location                 |
| Country     | VARCHAR(50)  | Country location              |
| Description | TEXT         | Detailed description          |
| AdminId     | INT (FK)     | Hotel administrator reference |

User Table

| Field Name      | Data type    | Data Description                                 |
|-----------------|--------------|--------------------------------------------------|
| User ID         | INT (PK)     | Unique identifier for user                       |
| User First Name | VARCHAR(50)  | First name of the user                           |
| User Last Name  | VARCHAR(20)  | Last name of the user                            |
| Email           | VARCHAR(200) | E-mail of the user for site                      |
| Phone Number    | INT          | Phone number of user                             |
| UserType        | VARCHAR(20)  | Type of user (Customer, HotelAdmin, SystemAdmin) |
| CreatedDate     | DATETIME     | Account creation date                            |
| LastLoginDate   | DATETIME     | Most recent login date                           |

Reservation Table:

| Field Name      | Data type     | Data Description                  |
|-----------------|---------------|-----------------------------------|
| ReservationId   | INT (PK)      | Unique identifier for reservation |
| UserId          | INT (FK)      | Reference to User table           |
| RoomId          | INT (FK)      | Reference to Room table           |
| CheckInDate     | DATE          | Arrival date                      |
| CheckOutDate    | DATE          | Departure date                    |
| TotalPrice      | DECIMAL(10,2) | Total cost of stay                |
| Status          | VARCHAR(20)   | Reservation status                |
| CreatedDate     | DATETIME      | When reservation was made         |
| SpecialRequests | TEXT          | Customer's special requests       |

Room Table:

| Field Name  | Data type     | Data Description            |
|-------------|---------------|-----------------------------|
| Room ID     | INT (PK)      | Unique identifier for Room  |
| Hotel ID    | INT (FK)      | Reference to the Hotel's PK |
| Room Type   | VARCHAR(50)   | Type of room                |
| Picture     | VARCHAR(50)   | Hold the name of picture    |
| Capacity    | BOOL          | Capacity of the Room        |
| Price       | DECIMAL(10,2) | Price per night             |
| Room Number | INT           | Room Number                 |
| Description | TEXT          | Room description            |
| IsAvailable | BOOLEAN       | Current availability status |

Payment Table:

| Field Name    | Data type     | Data Description               |
|---------------|---------------|--------------------------------|
| PaymentId     | INT (FK)      | Unique identifier for payment  |
| ReservationId | INT (PK)      | Reference to Reservation table |
| Amount        | DECIMAL(10,2) | Payment amount                 |
| PaymentDate   | DATETIME      | When payment was processed     |
| PaymentMethod | VARCHAR(50)   | Method of payment              |
| TransactionId | VARCHAR(100)  | Payment gateway transaction ID |
| Status        | VARCHAR(20)   | Payment status                 |

Feedback Table:

| Field Name     | Data type   | Data Description                              |
|----------------|-------------|-----------------------------------------------|
| FeedbackId     | INT (PK)    | Unique identifier for feedback                |
| UserId         | INT (FK)    | Reference to User table                       |
| HotelId        | INT (FK)    | Reference to Hotel table                      |
| Rating         | INT         | Star rating (1-5)                             |
| Comment        | TEXT        | Written review                                |
| SubmissionDate | DATETIME    | When feedback was submitted                   |
| Status         | VARCHAR(20) | Feedback status (approved, pending, rejected) |

Support Table:

| Field Name  | Data type    | Data Description                     |
|-------------|--------------|--------------------------------------|
| TicketId    | INT (PK)     | Unique identifier for support ticket |
| UserId      | INT (FK)     | Reference to User table              |
| Subject     | VARCHAR(100) | Ticket subject                       |
| Message     | TEXT         | Customer's message                   |
| CreatedDate | DATETIME     | When ticket was created              |
| Status      | VARCHAR(50)  | Ticket status                        |
| Priority    | VARCHAR(20)  | Priority level                       |

## 5. Component Design

### 5.1 Controllers

#### 5.1.1 AuthController

**Type:** Controller

**Purpose:** Provides functionality for user registration, login, and password management.

**Component Interface:**

- Input: User credentials, registration details
  - Output: Success/failure messages, redirect to appropriate pages •
- Dependencies: UserService, Auth0Service

##### 5.1.1.1 Login Action

**Purpose:** Authenticate user

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Login(email, password):
    IF Request.Method == "GET"
    THEN
        RETURN View()
    ENDIF
    result ← UserService.AuthenticateUser(email, password) IF
    result.Success THEN
        CreateAuthCookie(result.UserInfo)
        RETURN RedirectToAction("Index",
        "Home")
    ELSE
        ModelState.AddError("Authentication failed: " + result.ErrorMessage)
    RETURN View() ENDIF
END
FUNCTION
N
```

##### 5.1.1.2 Register Action

**Purpose:** Register a new user

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Register(registerViewModel): IF
    Request.Method == "GET" THEN
        RETURN
    View() ENDIF
```

END

```
I      State.IsValid THEN RETURN
F      View(registerViewModel)
      ENDIF
N      result ← UserService.RegisterUser(registerViewModel) IF result.Success
O      THEN
T      RETURN RedirectToAction("Login", "Auth", { message = "Registration
successful" }) ELSE
M      ModelState.AddModelError("Registration failed: " + result.ErrorMessage) RETURN
o      View(registerViewModel)
d      ENDIF
e
l
FUNCTION
```

### 5.1.1.3 ChangePassword Action

**Purpose:** Allow users to change their password

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION ChangePassword(changePasswordViewModel): IF
    Request.Method == "GET" THEN
        RETURN
    View() ENDIF

    IF NOT ModelState.IsValid THEN
        RETURN View(changePasswordViewModel) ENDIF

    userId ← GetCurrentUserId()
    result ← UserService.ChangePassword(userId, changePasswordViewModel.CurrentPassword,
    changePasswordViewModel.NewPassword)

    IF result.Success THEN
        RETURN RedirectToAction("Index", "UserDashboard", { message = "Password changed
    successfully" }) ELSE
        ModelState.AddError("Password change failed: " + result.ErrorMessage)
        RETURN View(changePasswordViewModel)
    ENDIF

EN
D
FUNCTION
```

### 5.1.2 HotelController

**Type:** Controller

**Purpose:** Manages hotel listings, search, and details.

**Component Interface:**

- Input: Search criteria, filter options
- Output: Hotel listings, detailed hotel information
- Dependencies: HotelService, RoomService, FeedbackService

#### 5.1.2.1 Index Action Purpose:

Display list of hotels

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Index(page = 1, pageSize = 10): hotelListViewModel ← NEW
    HotelListViewModel

    totalHotels ← HotelService.GetTotalHotelCount() hotels ←
    HotelService.GetHotels(page, pageSize)

    hotelListViewModel.Hotels ← hotels
    hotelListViewModel.Pagination ← NEW PaginationInfo(page, pageSize, totalHotels)

    RETURN View(hotelListViewModel) END
FUNCTION
```

**5.1.2.2 Search Action**

**Purpose:** Search for hotels based on criteria

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Search(searchViewModel): IF NOT
    ModelState.IsValid THEN
        RETURN RedirectToAction("Index") ENDIF

    searchResults ← HotelService.SearchHotels(
        searchViewModel.Location,
        searchViewModel.CheckInDate,
        searchViewModel.CheckOutDate,
        searchViewModel.Guests,
        searchViewModel.Page,
        searchViewModel.PageSize
    )

    resultViewModel ← NEW SearchResultViewModel
    resultViewModel.Hotels ← searchResults.Hotels
    resultViewModel.TotalResults ← searchResults.TotalCount
    resultViewModel.SearchCriteria ← searchViewModel
    resultViewModel.Pagination ← NEW PaginationInfo(searchViewModel.Page, searchViewModel.PageSize,
    searchResults.TotalCount)

    RETURN View(resultViewModel) END
FUNCTION
```

### 5.1.2.3 Details Action

**Purpose:** Display detailed hotel information

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Details(id):
    hotel ← HotelService.GetHotelById(id)

    IF hotel IS NULL THEN
        RETURN
    NotFound() ENDIF

    rooms ← RoomService.GetRoomsByHotel(id)
    reviews ← FeedbackService.GetFeedbackByHotel(id, 1, 5) // First 5 reviews

    detailsViewModel ← NEW HotelDetailsViewModel
    detailsViewModel.Hotel ← hotel detailsViewModel.Rooms ←
    rooms detailsViewModel.Reviews ← reviews
    detailsViewModel.AverageRating ← FeedbackService.GetAverageRating(id)

    RETURN View(detailsViewModel) END
FUNCTION
```

### 5.1.2.4 Filter Action

**Purpose:** Filter hotel search results

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Filter(filterViewModel): IF NOT
    ModelState.IsValid THEN
        RETURN RedirectToAction("Search", filterViewModel.SearchCriteria) ENDIF

    filteredResults ← HotelService.FilterHotels(
        filterViewModel.SearchCriteria,
        filterViewModel.PriceMin,
        filterViewModel.PriceMax,
        filterViewModel.StarRating,
        filterViewModel.Amenities,
        filterViewModel.SortBy, filterViewModel.Page,
        filterViewModel.PageSize
    )

    resultViewModel ← NEW SearchResultViewModel resultViewModel.Hotels ←
    filteredResults.Hotels resultViewModel.TotalResults ← filteredResults.TotalCount
    resultViewModel.SearchCriteria ← filterViewModel.SearchCriteria
    resultViewModel.FilterCriteria ← filterViewModel
    resultViewModel.Pagination ← NEW PaginationInfo(filterViewModel.Page, filterViewModel.PageSize,
    filteredResults.TotalCount)

    RETURN View("Search",
    resultViewModel) END FUNCTION
```



### 5.1.3 ReservationController

**Type:** Controller

**Purpose:** Manages the booking process and reservation management.

**Component Interface:**

- **Input:** Reservation details, room selection, payment information
- **Output:** Booking confirmation, reservation status updates
- **Dependencies:** ReservationService, RoomService, PaymentService, EmailService

#### 5.1.3.1 Create Action

**Purpose:** Create a new reservation

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Create(roomId, checkInDate, checkOutDate, guests):  
    IF NOT IsAuthenticated() THEN  
  
        RETURN RedirectToAction("Login", "Auth", { returnUrl = CurrentUrl }) ENDIF  
  
    room ← RoomService.GetRoomById(roomId)  
  
    IF room IS NULL THEN  
        RETURN  
        NotFound() ENDIF  
  
    IF NOT RoomService.IsRoomAvailable(roomId, checkInDate, checkOutDate) THEN RETURN  
        RedirectToAction("Details", "Hotel", { id = room.HotelId, error =  
"Room no longer available" }) ENDIF  
    totalPrice ← ReservationService.CalculatePrice(roomId, checkInDate,  
checkOutDate) bookingViewModel ← NEW BookingViewModel  
    bookingViewModel.Room ← room  
    bookingViewModel.CheckInDate ←  
checkInDate bookingViewModel.CheckOutDate  
← checkOutDate bookingViewModel.Guests ←  
guests bookingViewModel.TotalPrice ←  
totalPrice  
  
    RETURN View(bookingViewModel) END  
FUNCTION
```

### 5.1.3.2 Confirm Action

**Purpose:** Confirm reservation details before payment

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Confirm(bookingViewModel):
    IF NOT ModelState.IsValid THEN
        RETURN View("Create", bookingViewModel) ENDIF

    // Re-validate availability
    IF NOT RoomService.IsRoomAvailable(bookingViewModel.RoomId, bookingViewModel.CheckInDate,
    bookingViewModel.CheckOutDate) THEN
        ModelState.AddError("Room is no longer available for selected dates") RETURN View("Create",
        bookingViewModel)

    ENDIF

    // Re-calculate price to ensure accuracy bookingViewModel.TotalPrice ←
    ReservationService.CalculatePrice(
        bookingViewModel.RoomId,
        bookingViewModel.CheckInDate,
        bookingViewModel.CheckOutDate
    )
    RETURN View(bookingViewModel) END
FUNCTION
```

### 5.1.3.3 Payment Action

**Purpose:** Process payment for reservation

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION
    Payment(bookingViewModel): IF
    NOT ModelState.IsValid THEN
        RETURN View("Confirm", bookingViewModel) ENDIF

    // Create temporary reservation
    reservationId ← ReservationService.CreateReservation( GetCurrentUser(),
        bookingViewModel.RoomId,
        bookingViewModel.CheckInDate,
        bookingViewModel.CheckOutDate,
        bookingViewModel.Guests,
        bookingViewModel.TotalPrice,
        "Pending"
    )
    // Prepare payment model paymentViewModel ← NEW
    PaymentViewModel
    paymentViewModel.ReservationId ← reservationId paymentViewModel.Amount
    ← bookingViewModel.TotalPrice paymentViewModel.Currency ← "USD"
    paymentViewModel.ReturnUrl ← GeneratePaymentReturnUrl(reservationId)

    RETURN View(paymentViewModel) END
FUNCTION
```

**PaymentComplete Action Purpose:** Handle payment completion

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION PaymentComplete(reservationId, status, transactionId): reservation ←  
    ReservationService.GetReservationById(reservationId)  
  
    IF reservation IS NULL THEN RETURN  
        NotFound()  
    ENDIF  
  
    IF status == "success" THEN  
        // Update reservation status ReservationService.UpdateReservationStatus(reservationId, "Confirmed")  
  
        // Record payment  
        PaymentService.RecordPayment(reservationId, reservation.TotalPrice, "Credit Card", transactionId,  
            "Completed")  
        // Send confirmation email  
        EmailService.SendReservationConfirmation(reservation) RETURN  
        RedirectToAction("Success", new { id = reservationId }) ELSE  
        // Handle failed payment ReservationService.UpdateReservationStatus(reservationId, "PaymentFailed")  
  
        RETURN RedirectToAction("Failed", new { id = reservationId, error = "Payment processing  
        failed" }) ENDIF  
    END  
FUNCTION
```

#### 5.1.3.4 Cancel Action

**Purpose:** Cancel an existing reservation

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Cancel(id):  
    reservation ← ReservationService.GetReservationById(id)  
  
    IF reservation IS NULL THEN RETURN  
        NotFound()  
    ENDIF  
    IF reservation.UserId != GetCurrentUserId() AND NOT IsAdmin() THEN  
        RETURN Forbidden()  
    ENDIF  
    IF Request.Method == "GET"  
        THEN RETURN  
        View(reservation)  
    ENDIF  
    result ← ReservationService.CancelReservation(id) IF result.Success  
    THEN  
        // Send cancellation email EmailService.SendCancellationConfirmation(reservation)  
  
        RETURN RedirectToAction("Index", "UserDashboard", { message = "Reservation cancelled  
        successfully" }) ELSE  
        RETURN View(reservation, { error = result.ErrorMessage })  
    ENDIF END FUNCTION
```

#### 5.1.4 HotelManagementController

**Type:** Controller

**Purpose:** Provides hotel administration functionality.

**Component Interface:**

- Input: Hotel and room details, availability updates
- Output: Management dashboards, confirmation messages
- Dependencies: HotelService, RoomService, ReservationService

##### 5.1.4.1 Dashboard Action

**Purpose:** Display hotel admin dashboard

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION Dashboard():
    IF NOT IsHotelAdmin() THEN

        RETURN RedirectToAction("Login", "Auth") ENDIF

    adminId ← GetCurrentUserId()
    hotels ← HotelService.GetHotelsByAdminId(adminId)

    dashboardViewModel ← NEW HotelAdminDashboardViewModel
    dashboardViewModel.Hotels ← hotels

    IF hotels.Count == 1 THEN
        // If admin has only one hotel, load detailed stats hotel ←
        hotels[0] dashboardViewModel.CurrentBookings ←
        ReservationService.GetActiveReservationsByHotel(hotel.Id) dashboardViewModel.RecentReviews ←
        FeedbackService.GetRecentFeedbackByHotel(hotel.Id, 5) dashboardViewModel.OccupancyRate ←
        RoomService.GetOccupancyRate(hotel.Id) dashboardViewModel.RevenueStats ←
        ReservationService.GetRevenueStats(hotel.Id)
    ENDIF

    RETURN
    View(dashboardViewModel) END
FUNCTION
```

##### 5.1.4.2 AddRoom Action Purpose:

Add a new room to a hotel

**Type:** Controller Action

**Pseudocode:**

```

FUNCTION AddRoom(hotelId):
    IF NOT CanManageHotel(hotelId) THEN
        RETURN Forbidden()
    ENDIF
    hotel ← HotelService.GetHotelById(hotelId) IF hotel IS
    NULL THEN
        RETURN
    NotFound() ENDIF

    IF Request.Method == "GET" THEN roomViewModel
        ← NEW RoomViewModel
        roomViewModel.HotelId ← hotelId RETURN
        View(roomViewModel)
    ENDIF

    IF NOT ModelState.IsValid THEN
        RETURN View(roomViewModel)
    ENDIF

    roomId ←
        RoomService.AddRoom(
            roomViewModel.HotelId,
            roomViewModel.RoomNum
            ber,
            roomViewModel.RoomType,
            roomViewModel.Capacity,
            roomViewModel.Price,
            roomViewModel.Description
            ,
            roomViewModel.Amenities,
            roomViewModel.IsAvailable
        )

    // Handle image uploads if any
    IF roomViewModel.Images IS NOT NULL AND roomViewModel.Images.Count > 0 THEN
        FOREACH image IN roomViewModel.Images DO
            RoomService.AddRoomImage(roomId,
            image) END FOREACH
    ENDIF

    RETURN RedirectToAction("Rooms", new { hotelId = hotelId, message = "Room added
successfully" }) END FUNCTION

```

#### 5.1.4.3 EditRoom Action

**Purpose:** Modify room details

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION EditRoom(id):
    room ← RoomService.GetRoomById(id)

    IF room IS NULL THEN
        RETURN
    NotFound() ENDIF

    IF NOT CanManageHotel(room.HotelId) THEN
        RETURN Forbidden()
    ENDIF

    IF Request.Method == "GET" THEN roomViewModel ←
        MapRoomToViewModel(room) RETURN
        View(roomViewModel)
    ENDIF

    IF NOT ModelState.IsValid THEN
        RETURN View(roomViewModel)
    ENDIF

    result ← RoomService.UpdateRoom( id,
        roomViewModel.RoomNumber,
        roomViewModel.RoomType,
        roomViewModel.Capacity,
        roomViewModel.Price,
        roomViewModel.Description,
        roomViewModel.Amenities,
        roomViewModel.IsAvailable
    )

    // Handle image uploads if any
    IF roomViewModel.Images IS NOT NULL AND roomViewModel.Images.Count > 0 THEN
        FOREACH image IN roomViewModel.Images DO
            RoomService.AddRoomImage(id, image)
        END FOREACH
    ENDIF

    RETURN RedirectToAction("Rooms", new { hotelId = room.HotelId, message = "Room updated
successfully" }) END FUNCTION
```

#### 5.1.4.4 DeleteRoom Action Purpose:

Remove a room from a hotel

**Type:** Controller Action

**Pseudocode:**

```

FUNCTION DeleteRoom(id):
room ← RoomService.GetRoomById(id)

IF room IS NULL THEN
    RETURN
    NotFound() ENDIF

IF NOT CanManageHotel(room.HotelId) THEN
    RETURN Forbidden()
ENDIF

// Check if room has active reservations
hasActiveReservations ← ReservationService.HasActiveReservations(id)

IF hasActiveReservations THEN
    RETURN RedirectToAction("Rooms", new { hotelId = room.HotelId, error = "Cannot delete
room with active reservations" })
ENDIF

hotelId ← room.HotelId // Save before deletion

RoomService.DeleteRoom(id)

RETURN RedirectToAction("Rooms", new { hotelId = hotelId, message
= "Room deleted successfully" }) END
FUNCTION

```

### 5.1.5 FeedbackController

**Type:** Controller

**Purpose:** Manages user reviews and ratings.

**Component Interface:**

- Input: User ratings and reviews
- Output: Confirmation messages, feedback listings
- Dependencies: FeedbackService, HotelService, ReservationService

#### 5.1.5.1 Submit Action

**Purpose:** Submit a new review for a hotel

## Type: Controller Action

### Pseudocode:

```
FUNCTION Submit(hotelId):
    IF NOT IsAuthenticated() THEN
        RETURN RedirectToAction("Login", "Auth", { returnUrl = CurrentUrl }) ENDIF

    userId ← GetCurrentUserId()

    // Check if user has stayed at this hotel
    hasStayed ← ReservationService.HasUserStayedAtHotel(userId, hotelId)

    IF NOT hasStayed AND NOT IsAdmin() THEN
        RETURN RedirectToAction("Details", "Hotel", { id = hotelId, error = "You can only review hotels
at"
}) ENDIF
    hotel ← HotelService.GetHotelById(hotelId) IF hotel IS
    NULL THEN
        RETURN
    NotFound() ENDIF

    IF Request.Method == "GET" THEN feedbackViewModel
        ← NEW FeedbackViewModel
        feedbackViewModel.HotelId ← hotelId
        feedbackViewModel.HotelName ← hotel.Name

        RETURN View(feedbackViewModel)
    ENDIF

    IF NOT ModelState.IsValid THEN
        feedbackViewModel.HotelName ← hotel.Name
        RETURN View(feedbackViewModel)
    ENDIF

    result ← FeedbackService.SubmitFeedback( userId,
        hotelId, feedbackViewModel.Rating,
        feedbackViewModel.Comment
    )
```



```

IF result.Success THEN
    RETURN RedirectToAction("Details", "Hotel", { id = hotelId, message = "Thank you for your
feedback" }) ELSE
    ModelState.AddError(result.ErrorMessage)

    feedbackViewModel.HotelName ← hotel.Name RETURN
    View(feedbackViewModel)
ENDIF
END
FUNCTION

```

### 5.1.6

#### **AdminController**

**Type:** Controller

**Purpose:** Provides system administration functions.

#### **Component Interface:**

- Input: Admin commands, approval decisions
- Output: Admin dashboards, user and hotel management interfaces
- Dependencies: UserService, HotelService, ReservationService, FeedbackService, SupportService

#### **5.1.6.1 Dashboard Action**

**Purpose:** Display system admin dashboard

**Type:** Controller Action

#### **Pseudocode:**

```

FUNCTION Dashboard():
    IF NOT IsSystemAdmin() THEN
        RETURN RedirectToAction("Login", "Auth") ENDIF

    dashboardViewModel ← NEW AdminDashboardViewModel
    dashboardViewModel.UserCount ← UserService.GetTotalUserCount()
    dashboardViewModel.HotelCount ← HotelService.GetTotalHotelCount()
    dashboardViewModel.ReservationCount ←
ReservationService.GetTotalReservationCount() dashboardViewModel.PendingHotels
← HotelService.GetPendingHotels() dashboardViewModel.RecentUsers ←
UserService.GetRecentUsers(5) dashboardViewModel.OpenTickets ←
SupportService.GetOpenTickets(5)

    RETURN View(dashboardViewModel)
END FUNCTION

```

### 5.1.6.2 ApproveHotel Action

**Purpose:** Approve or reject a hotel listing

**Type:** Controller Action

**Pseudocode:**

```
FUNCTION ApproveHotel(id, approved): IF
    NOT IsSystemAdmin() THEN
        RETURN RedirectToAction("Login", "Auth") ENDIF

    hotel ← HotelService.GetHotelById(id)

    IF hotel IS NULL THEN
        RETURN
        NotFound() ENDIF

    IF approved THEN
        result ← HotelService.ApproveHotel(id) message ← "Hotel
        approved
        successfully" ELSE
        result ← HotelService.RejectHotel(id) message ←
        "Hotel
        rejected" ENDIF

    IF result.Success THEN
        // Notify hotel admin via email
        adminUser ← UserService.GetUserById(hotel.AdminId)
        EmailService.SendHotelApprovalNotification(adminUser.Email,
        hotel.Name,
        approved)

        RETURN RedirectToAction("PendingHotels", new { message = message })
        ELSE RETURN RedirectToAction("PendingHotels", new { error =
        result.ErrorMessage
    })
    ENDIF
END FUNCTION
```

## 5.2 Models

### 5.2.1 User Model

**Type:** Model

**Purpose:** Represents user information in the system.

**Properties:**

- UserId: int
- FirstName: string
- LastName: string
- Email: string
- PhoneNumber: string
- UserType: enum (Customer, HotelAdmin, SystemAdmin)
- CreatedDate: DateTime
- LastLoginDate: DateTime

**Methods:**

```
FUNCTION IsInRole(role): RETURN  
    UserType == role
```

```
END FUNCTION
```

```
FUNCTION GetFullName():  
    RETURN FirstName + " " + LastName END  
FUNCTION
```

**5.2.2 Hotel Model**

**Type:** Model

**Purpose:**

Represents hotel information.

**Properties:**

- HotelId: int
- Name: string
- Address: string
- City: string
- Country: string
- Rating: int
- Description: string
- Amenities: List<string>
- Photos: List<string>
- AdminId: int
- Status: enum (Pending, Approved, Rejected)
- Rooms: List<Room>
- Reviews: List<Feedback>

**Methods:**

```

FUNCTION GetAverageRating():
    IF Reviews IS NULL OR Reviews.Count == 0 THEN
        RETURN 0 ENDIF

    totalRating ← 0

    FOREACH review IN Reviews DO
        totalRating ← totalRating + review.Rating END FOREACH

    RETURN totalRating / Reviews.Count END
FUNCTION

```

```

FUNCTION GetLowestRoomPrice():

    IF Rooms IS NULL OR Rooms.Count == 0 THEN RETURN 0

    ENDIF
    lowestPrice ← Rooms[0].Price FOREACH
    room IN Rooms DO
        IF room.Price < lowestPrice THEN
            lowestPrice ← room.Price
        ENDIF
    END
    FOREACH

    RETURN
    lowestPrice END
FUNCTION

```

### 5.2.3 Room Model

**Type:** Model

**Purpose:** Represents room information within a hotel.

**Properties:**

- RoomId: int
- HotelId: int
- RoomNumber: string
- RoomType: string
- Capacity: int
- Price: decimal
- Description: string
- Amenities: List<string>
- Photos: List<string>
- IsAvailable: bool
- Hotel: Hotel (navigation property)
- Reservations: List<Reservation> (navigation property)

**Methods:**

```

FUNCTION IsAvailableForDates(checkInDate, checkOutDate): IF
    NOT IsAvailable TH

```

```
    RETURN  
FALSE ENDIF
```

```
FOREACH reservation IN Reservations DO
```

```
    // Check if there's overlap between requested dates and existing reservation IF (checkInDate <=  
    reservation.CheckOutDate AND checkOutDate >=  
reservation.CheckInDate) THEN  
        RETURN  
    FALSE ENDIF  
END FOREACH
```

```
RETURN  
TRUE END  
FUNCTION
```

## 5.3 Services

### 5.3.1 UserService

**Type:** Service

**Purpose:** Handles user-related business logic.

#### 5.3.1.1 AuthenticateUser Method

**Purpose:** Validate user credentials

**Type:** Business Logic

**Pseudocode:**

```
FUNCTION AuthenticateUser(email, password): user ←  
    UserRepository.GetByEmail(email)  
  
    IF user IS NULL THEN  
        RETURN new AuthResult { Success = FALSE, ErrorMessage = "Invalid email or  
password" } ENDIF  
    passwordValid ← VerifyPassword(password, user.PasswordHash) IF NOT  
passwordValid THEN  
        RETURN new AuthResult { Success = FALSE, ErrorMessage = "Invalid email or password" }  
  
    ENDIF  
    // Update last login date user.LastLoginDate ←  
    CurrentDateTime() UserRepository.Update(user)  
  
    RETURN new AuthResult { Success  
        = TRUE,  
        UserInfo = new UserInfo { UserId =  
            user.UserId, Email = user.Email,  
            FirstName = user.FirstName,  
            UserType  
            = user.UserType  
        }  
    }  
END FUNCTION
```

#### 5.3.1.2 RegisterUser Method

**Purpose:** Create a new user account

**Type:** Business Logic

**Pseudocode:**

```

FUNCTION RegisterUser(registerModel):
    // Check if email already exists
    existingUser ← UserRepository.GetByEmail(registerModel.Email)

    IF existingUser IS NOT NULL THEN
        RETURN new RegistrationResult { Success = FALSE, ErrorMessage = "Email already
in use" } ENDIF

    // Create new user
    passwordHash ← HashPassword(registerModel.Password)

    newUser ← new User {
        FirstName = registerModel.FirstName, LastName
        = registerModel.LastName, Email =
        registerModel.Email, PhoneNumber =
        registerModel.PhoneNumber, PasswordHash =
        passwordHash,

        UserType = "Customer", // Default role CreatedDate =
        CurrentDateTime(), LastLoginDate = NULL
    }

    userId ← UserRepository.Add(newUser)

    RETURN new RegistrationResult { Success = TRUE, UserId = userId } END FUNCTION

```

### 5.3.2 HotelService

**Type:** Service

**Purpose:** Manages hotel-related business logic.

#### 5.3.2.1 SearchHotels Method

**Purpose:** Search for hotels based on criteria

**Type:** Business Logic

**Pseudocode:**

```

FUNCTION SearchHotels(location, checkInDate, checkOutDate, guests, page, pageSize):
    // Validate parameters
    IF location IS NULL OR location == "" THEN RETURN new
    SearchResult {
        Success = FALSE,
        ErrorMessage = "Location is required"
    } ENDIF

    IF checkInDate <= CurrentDate() THEN
        RETURN new SearchResult {
            Success = FALSE,
            ErrorMessage = "Check-in date must be in the future"
        } ENDIF

    IF checkOutDate <= checkInDate THEN
        RETURN new SearchResult {
            Success = FALSE,
            ErrorMessage = "Check-out date must be after check-in date"
        } ENDIF

```

```

        // Try to get from cache first
cacheKey ← GenerateCacheKey(location, checkInDate, checkOutDate, guests,
page, pageSize) cachedResult ← CacheService.Get(cacheKey)

IF cachedResult IS NOT NULL
    THEN RETURN
    cachedResult
ENDIF

// Search database
hotels ← HotelRepository.SearchHotels(location, page, pageSize) totalCount ←
HotelRepository.GetSearchResultCount(location)

// Filter by room availability availableHotels ← []

FOREACH hotel IN hotels DO
    availableRooms ← RoomRepository.GetAvailableRooms( hotel.HotelId,
        checkInDate,
        checkOutDate,
        guests
    )

    IF availableRooms.Count > 0 THEN hotel.AvailableRooms ←
        availableRooms ADD hotel TO
        availableHotels ENDIF
END FOREACH

result ← new SearchResult { Success =
    TRUE,
    Hotels = availableHotels, TotalCount =
    totalCount
}

// Cache the result
CacheService.Set(cacheKey, result, TimeSpan.FromMinutes(5))

RETURN
result END
FUNCTION

```

### 5.3.3 ReservationService

**Type:** Service

**Purpose:** Handles booking-related business logic.

#### 5.3.3.1 CreateReservation Method

**Purpose:** Create a new booking

**Type:** Business Logic

**Pseudocode:**

```
FUNCTION CreateReservation(userId, roomId, checkInDate, checkOutDate, guests, totalPrice, status):
    // Validate room availability
    isAvailable ← RoomRepository.CheckAvailability(roomId, checkInDate, checkOutDate)

    IF NOT isAvailable THEN
        RETURN new ReservationResult {
            Success = FALSE,
            ErrorMessage = "Room is no longer available for the selected dates"
        } ENDIF

    // Create reservation newReservation ← new
    Reservation {
        UserId = userId,
        RoomId = roomId, CheckInDate =
        checkInDate, CheckOutDate =
        checkOutDate, Guests = guests,
        TotalPrice = totalPrice, Status =
        status, CreatedDate =
        CurrentDateTime()
    }

    reservationId ← ReservationRepository.Add(newReservation)

    // Mark room as unavailable for these dates RoomRepository.UpdateAvailabilityForDates(roomId,
    checkInDate, checkOutDate,
    FALSE)

    RETURN new ReservationResult { Success
        = TRUE, ReservationId = reservationId
    }
END FUNCTION
```



## 6. Human User Interface Design

### 6.1 Overview of User Interface

The Prestige Hotel Reservation system provides a responsive web interface that adapts to different screen sizes and devices. The interface follows modern design principles with a clean, intuitive layout that guides users through the booking process. It allows each type of user (Customer, Hotel Admin, System Admin) interact with the system efficiently. Search & Filter Panel for customers to search hotels by location, date, price, and rating. The user interface is organized into some main sections as Public Pages (home, search results, hotel details, login/register), Customer Dashboard (bookings, profile, reviews), Hotel Admin Panel (hotel management, booking, reports) and System Admin Panel (user management, hotel approvals, system settings).

### 6.2 Screen Images

**SCREEN IMAGES IS ATTACHED AT THE BOTTOM OF THE REPORT (Annex E: Screen images)**

Home Page: Features a main section with search functionality, popular destinations, and featured hotels  
Search Hotel Page: Displays hotels with filtering options, sorting capabilities, and detailed information cards  
Login Page: Simple forms for user authentication  
Create Account Page: Simple forms for account creation

Support Page: FAQ section and contact form for customer support

### 6.3 Screen Objects and Actions

Home Page Objects and Actions:

Search Form: Destination input field; Check-in/check-out date pickers; Guest count selector; Search button (triggers hotel search)

Featured Hotels Carousel: Hotel cards with images, ratings, and prices; "View Details" buttons (navigate to hotel details)

Navigation Menu: Home, Hotels, Destinations, Special Offers, Support links; Login/Register buttons (open authentication forms)

Search Results Page Objects and Actions:

Filter Panel: Price range slider; Star rating checkboxes; Facilities checkboxes; "Apply Filters" button (updates search results)  
Sort Dropdown: Options: Relevance, Price (low to high), Price (high to low), Rating; Changes result ordering immediately  
Hotel Cards: Hotel image, name, rating, price; "View Details" button (navigates to hotel details); Basic information display  
Hotel Details Page Objects and Actions:  
Image Gallery: Main image display; Thumbnail navigation; Fullscreen view option

Booking Widget: Room type selector; Date selection; Guest count; "Book Now" button (initiates booking process)  
Information Tabs: Overview, Rooms, Facilities, Reviews, Location; Tab switching updates displayed content  
Booking Process Objects and Actions:

Room Selection: Room type cards with details; "Select" buttons for each room type  
Guest Information Form: Name, email, phone fields; Special requests textarea

Payment Form: Credit card fields; Billing address inputs; "Complete Booking" button (processes payment)

7. Requirements Matrix

UC-1: Login System Requirements

Functional Requirements:

| Requirement ID | Requirement Description                                | Design Component                                            | Implementation Details                          |
|----------------|--------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------|
| UC1-REQ-1      | Authenticate users using registered email and password | AuthenticationService, Auth0Client, UserRepository          | Auth0 integration with JWT token generation     |
| UC1-REQ-2      | Display error message if login fails                   | LoginController, ErrorView<br>Model, Login.cshtml           | Client-side and server-side validation          |
| UC1-REQ-3      | Allow password reset via email                         | PasswordResetService, EmailService, ResetPasswordController | Token-based password reset flow                 |
| UC1-REQ-4      | Offer account registration for unregistered users      | RegistrationService, UserController, Register.cshtml        | Multi-step registration with email verification |
| UC1-REQ-5      | Allow access only if credentials are valid             | AuthorizationMiddleware, JwtTokenValidator                  | Role-based access control                       |
| UC1-REQ-6      | Redirect valid users to their homepage                 | LoginController, UserDashboardController                    | Role-specific redirect logic                    |
| UC1-REQ-7      | Display welcome message after successful login         | LoginService, SessionManager, Dashboard.cshtml              | Personalized welcome based on user type         |

Non-Functional Requirements:

| Requirement ID | Requirement Description                 | Design Component                  | Implementation Details                   |
|----------------|-----------------------------------------|-----------------------------------|------------------------------------------|
| UC1-NFR-1      | Process login requests within 2 seconds | PerformanceMonitor, CacheService  | Redis caching for user sessions          |
| UC1-NFR-2      | Support 500 concurrent login requests   | LoadBalancer, ConnectionPool      | Horizontal scaling with multiple servers |
| UC1-NFR-3      | Encrypt password entries                | PasswordHasher, TLS Configuration | BCrypt hashing with salt                 |
| UC1-NFR-4      | Record every login attempt for audit    | AuditLogger,                      | Asynchronous logging to                  |

UC-2: Manage Hotel Requirements

Functional Requirements

| Requirement ID | Requirement Description                         | Design Component                                               | Implementation Details                |
|----------------|-------------------------------------------------|----------------------------------------------------------------|---------------------------------------|
| UC2-REQ-1      | Allow hotel admins to add new hotels            | HotelManagementService , HotelRepository, CreateHotelValidator | Multi-step hotel creation wizard      |
| UC2-REQ-2      | List existing hotels under admin names          | HotelListService, HotelRepository, AdminDashboard              | Paginated list with search and filter |
| UC2-REQ-3      | Allow system admins to add hotels for any admin | SystemAdminService, HotelAssignmentService                     | Admin delegation functionality        |
| UC2-REQ-4      | Update hotel and room details                   | HotelUpdateService, RoomManagementService                      | Versioned updates with audit trail    |
| UC2-REQ-5      | Show error if hotel with same name exists       | HotelValidator, DuplicateCheckService                          | Real-time validation during input     |
| UC2-REQ-6      | Create promotional campaigns                    | CampaignService, PricingEngine                                 | Date-based campaign management        |
| UC2-REQ-7      | Update room availability in real-time           | RoomAvailabilityService WebSocketManager                       | WebSocket-based real-time updates     |
| UC2-REQ-8      | Set and update room pricing                     | PricingService, SeasonalPricingEngine                          | Dynamic pricing with rules engine     |

#### Non-Functional Requirements

| Requirement ID | Requirement Description | Design Component | Implementation Details |
|----------------|-------------------------|------------------|------------------------|
| UC2-NFR-1      | List hotels within 10   | QueryOptimizer,  | Indexed queries with   |

|                  |                                               |                                        |                                         |
|------------------|-----------------------------------------------|----------------------------------------|-----------------------------------------|
|                  | seconds                                       | DatabaseIndexes                        | pagination                              |
| <b>UC2-NFR-2</b> | Send confirmation email after cancellation    | NotificationService,<br>EmailQueue     | Asynchronous email processing           |
| <b>UC2-NFR-3</b> | Send confirmation email after adding hotel    | EmailService,<br>HotelCreationWorkflow | Template-based email system             |
| <b>UC2-NFR-4</b> | Send confirmation email after modifying hotel | NotificationService,<br>ChangeTracker  | Event-driven notifications              |
| <b>UC2-NFR-5</b> | Send confirmation email after removing hotel  | DeletionService,<br>EmailNotifier      | Soft delete with notification           |
| <b>UC2-NFR-6</b> | Reflect data updates in real-time             | EventBroadcaster,<br>SignalR Hub       | Push notifications to connected clients |

### UC-3: Manage Users Requirements

#### Functional Requirements

| Requirement ID   | Requirement Description                      | Design Component                                                | Implementation Details                     |
|------------------|----------------------------------------------|-----------------------------------------------------------------|--------------------------------------------|
| <b>UC3-REQ-1</b> | View complete list of registered users       | UserListService,<br>UserRepository,<br><br>AdminUserList.cshtml | Filterable and sortable user grid          |
| <b>UC3-REQ-2</b> | Edit user profiles                           | UserProfileService,<br>ProfileUpdateValidator                   | Field-level validation and change tracking |
| <b>UC3-REQ-3</b> | Reset user passwords with email notification | PasswordResetService,<br>EmailNotificationService               | Secure token generation and validation     |
| <b>UC3-REQ-4</b> | Validate privilege changes                   | PrivilegeValidator,<br>RoleManagementService                    | Permission matrix validation               |

Non-Functional Requirements

| Requirement ID | Requirement Description                       | Design Component                        | Implementation Details                   |
|----------------|-----------------------------------------------|-----------------------------------------|------------------------------------------|
| UC3-NFR-1      | Maintain audit log of user management actions | AuditService, UserActionLogger          | Immutable audit trail                    |
| UC3-NFR-2      | Encrypt user data in database                 | DataEncryptionService, ColumnEncryption | AES-256 encryption at rest               |
| UC3-NFR-3      | Complete interface actions within 2 seconds   | PerformanceOptimizer, ResponseCache     | Client-side caching and optimization     |
| UC3-NFR-4      | Send password reset emails within 1 minute    | EmailQueueService, BackgroundWorker     | Priority queue for time-sensitive emails |

UC-4: Reserve Hotel Requirements  
Functional Requirements

| Requirement ID | Requirement Description                    | Design Component                              | Implementation Details          |
|----------------|--------------------------------------------|-----------------------------------------------|---------------------------------|
| UC4-REQ-1      | View room availability for selected hotels | AvailabilityService, RoomInventoryManager     | Real-time availability checking |
| UC4-REQ-2      | Display room types and pricing             | PricingService, RoomTypeRepository            | Dynamic pricing display         |
| UC4-REQ-3      | Allow logged-in customers to reserve rooms | BookingService, ReservationValidator          | Authenticated booking flow      |
| UC4-REQ-4      | Confirm successful bookings                | BookingConfirmationService, ConfirmationView  | Confirmation page with details  |
| UC4-REQ-5      | Notify customer and admin after booking    | NotificationDistributor, MultiChannelNotifier | Email and in-app notifications  |

|            |                                      |                                                   |                                   |
|------------|--------------------------------------|---------------------------------------------------|-----------------------------------|
| UC4-REQ-6  | Log reservation details for security | ReservationLogger, SecurityAuditService           | Comprehensive booking audit trail |
| UC4-REQ-7  | Validate date selection              | DateValidator, BookingRulesEngine                 | Business rule validation          |
| UC4-REQ-8  | Update availability in real-time     | AvailabilityUpdater, InventorySynchronizer        | Immediate inventory updates       |
| UC4-REQ-9  | Validate payment method              | PaymentValidator, BankingService                  | Payment gateway integration       |
| UC4-REQ-10 | Auto-cancel incomplete payments      | PaymentTimeoutHandler, BookingCancellationService | Timeout-based cancellation        |
| UC4-REQ-11 | Display availability status          | AvailabilityIndicator, StatusRenderer             | Visual status indicators          |
| UC4-REQ-12 | Show latest pricing                  | DynamicPricingService, PriceCalculator            | Real-time price calculation       |
| UC4-REQ-13 | Apply active campaigns               | CampaignApplicator, DiscountEngine                | Automatic discount application    |

Non-Functional Requirements

| Requirement ID | Requirement Description                      | Design Component                        | Implementation Details       |
|----------------|----------------------------------------------|-----------------------------------------|------------------------------|
| UC4-NFR-1      | Support 100 concurrent bookings              | BookingQueueManager, TransactionManager | Queued booking processing    |
| UC4-NFR-2      | Send cancellation confirmation email         | CancellationNotifier, EmailService      | Template-based notifications |
| UC4-NFR-3      | Protect payment transactions with encryption | PaymentEncryptor, TLSConfiguration      | End-to-end encryption        |
| UC4-NFR-4      | Update room availability in real-time        | RealTimeUpdater, WebSocketService       | Push-based updates           |

UC-5: Search Hotel Requirements

Functional Requirements

| Requirement ID | Requirement Description                         | Design Component                           | Implementation Details    |
|----------------|-------------------------------------------------|--------------------------------------------|---------------------------|
| UC5-REQ-1      | Input search criteria (location, dates, guests) | SearchFormValidator, SearchCriteriaBuilder | Multi-field search form   |
| UC5-REQ-2      | Return matching hotels                          | SearchEngine, HotelMatcher                 | Elasticsearch integration |
| UC5-REQ-3      | Sort results by price                           | SortingService, PriceComparator            | Bidirectional sorting     |
| UC5-REQ-4      | Suggest alternatives if no results              | SuggestionEngine, AlternativeSearchService | ML-based suggestions      |
| UC5-REQ-5      | Search by hotel name                            | NameSearchService, TextSearchEngine        | Fuzzy name matching       |

Non-Functional Requirements

| Requirement ID | Requirement Description              | Design Component                         | Implementation Details          |
|----------------|--------------------------------------|------------------------------------------|---------------------------------|
| UC5-NFR-1      | Load search results within 3 seconds | SearchOptimizer, CacheManager            | Cached search results           |
| UC5-NFR-2      | Support 100 concurrent searches      | SearchLoadBalancer, SearchCluster        | Distributed search processing   |
| UC5-NFR-3      | Maintain 99.9% search availability   | HighAvailabilityManager, FailoverService | Redundant search infrastructure |
| UC5-NFR-4      | Responsive design for all devices    | ResponsiveUIFramework, AdaptiveLayout    | Mobile-first design             |
| UC5-NFR-5      | Cache recent searches                | SearchCacheService, UserSearchHistory    | Redis-based search caching      |

**UC-6: Give Feedback Requirements**  
**Functional Requirements**

| Requirement ID | Requirement Description               | Design Component                      | Implementation Details      |
|----------------|---------------------------------------|---------------------------------------|-----------------------------|
| UC6-NFR-1      | Display rating and feedback interface | RatingComponent,                      | Star rating with text       |
| UC6-NFR-2      | Update hotel page with new feedback   | FeedbackUpdater, HotelPageRenderer    | Asynchronous page updates   |
| UC6-NFR-3      | Moderate inappropriate content        | ContentModerator, ReviewFilterService | Automated content filtering |
| UC6-NFR-4      | Allow editing of reviews              | ReviewEditService, VersionController  | Versioned review editing    |
| UC6-NFR-5      | Prevent feedback for unvisited hotels | VisitValidator, BookingVerifier       | Booking-based validation    |

**Non-Functional Requirements**

| Requirement ID | Requirement Description                | Design Component                           | Implementation Details   |
|----------------|----------------------------------------|--------------------------------------------|--------------------------|
| UC6-NFR-1      | Update hotel page within 10 seconds    | PageUpdateService, CacheInvalidator        | Incremental page updates |
| UC6-NFR-2      | Send confirmation email after feedback | FeedbackNotifier, EmailConfirmationService | Automated email workflow |
| UC6-NFR-3      | Reflect updates in real-time           | RealTimeFeedbackService, WebSocketUpdater  | Live feedback updates    |

**Performance Requirements Matrix**



| Requirement ID | Performance Metric        | Design Component                        | Implementation Details |
|----------------|---------------------------|-----------------------------------------|------------------------|
| PERF-1         | Database update frequency | DatabaseSynchronizer, ChangeDetector    | Every 3 seconds        |
| PERF-2         | Query result load time    | QueryOptimizer, ResultCache             | < 4 seconds            |
| PERF-3         | UI load time              | FrontendOptimizer, AssetBundler         | < 2 seconds            |
| PERF-4         | Login validation time     | AuthenticationValidator, SessionManager | < 3 seconds            |
| PERF-5         | Concurrent user support   | LoadBalancer, ServerCluster             | 500+ users             |
| PERF-6         | Payment confirmation time | PaymentProcessor, TransactionManager    | < 5 seconds            |

Security Requirements Matrix

| Requirement ID | Security Measure          | Design Component                         | Implementation    |
|----------------|---------------------------|------------------------------------------|-------------------|
| SEC-1          | Client-server encryption  | TLSConfiguration, CertificateManager     | TLS 1.3           |
| SEC-2          | Data protection           | DataEncryptionService, KeyManager        | AES-256           |
| SEC-3          | Database backup           | BackupService, RecoveryManager           | Hourly backups    |
| SEC-4          | System recovery           | DisasterRecoveryService, FailoverManager | 1-hour RTO        |
| SEC-5          | Admin data access control | PermissionManager, DataMaskingService    | Role-based access |
| SEC-6          | 3D Secure payments        | PaymentGateway, SecurePaymentValidator   | 3DS 2.0           |

|       |                       |                                           |                 |
|-------|-----------------------|-------------------------------------------|-----------------|
| SEC-7 | Password requirements | PasswordPolicyEnforcer, ValidationService | 8+ alphanumeric |
|-------|-----------------------|-------------------------------------------|-----------------|

### Implementation Priority

| Priority | Use Case                | Justification                                      |
|----------|-------------------------|----------------------------------------------------|
| High     | UC-1 (Login)            | Core functionality required for all other features |
| High     | UC-4 (Booking)          | Primary business function                          |
| High     | UC-5 (Search)           | Essential for user experience                      |
| Medium   | UC-2 (Hotel Management) | Administrative function                            |
| Medium   | UC-6 (Feedback)         | Enhanced user experience                           |
| Low      | UC-3 (User Management)  | Administrative support function                    |

## 8. Appendices

### Annex A: Bibliography

The following references were used in the development of this Software Design Document:

1. IEEE Std 1016-2009, IEEE Standard for Information Technology—Systems Design—Software Design Descriptions, IEEE Computer Society, 2009.
2. Microsoft Documentation, ASP.NET Core MVC Overview, <https://docs.microsoft.com/en-us/aspnet/core/mvc/overview>,
3. Auth0 Documentation, Authentication and Authorization Fundamentals, <https://auth0.com/docs/get-started/authentication-and-authorization>,
4. Redis Documentation, Introduction to Redis, <https://redis.io/documentation>,
5. Diagram help eraser.io <https://www.eraser.io/ai/architecture-diagram-generator>

### Annex B: Design Language Description

This annex defines the notations and conventions used throughout this Software Design Document.

#### B.1 Unified Modeling Language (UML)

The design diagrams in this document use UML 2.5 notation with the following conventions:

## **Class Diagrams:**

Classes are represented as rectangles with three compartments (name, attributes, methods)

Relationships use standard UML notation:

- Association: solid line
- Inheritance: hollow triangle arrowhead
- Composition: filled diamond
- Aggregation: hollow diamond
- Dependency: dashed arrow

## **Sequence Diagrams:**

- Lifelines are shown as vertical dashed lines
- Messages are horizontal arrows between lifelines
- Synchronous calls: filled arrowhead
- Asynchronous calls: open arrowhead
- Return messages: dashed lines

## **Component Diagrams:**

- Components are rectangles with component icon
- Interfaces are circles (provided) or semicircles (required)
- Dependencies are dashed arrows

## **B.2 Entity-Relationship Notation**

Database designs use the following conventions:

- Entities: rectangles

- Attributes: listed within entity rectangles
- Primary Keys: marked with (PK)
- Foreign Keys: marked with (FK)
- Relationships: lines with cardinality notation (1, N, 0..1, etc.)

### **B.3 Architecture Diagrams**

System architecture diagrams use:

- Layers: horizontal rectangles
- Components: rounded rectangles
- External systems: cloud shapes
- Data flow: arrows with labels
- APIs: lollipop notation

## **Annex C: Software Design Document Template**

This template conforms to IEEE Std 1016-2009 and was used to organize this SDD.

### **C.1 Front Matter**

#### **1. Title Page**

- Document title
- Project name
- Version number
- Date
- Author information

#### **2. Revision History**

- Version tracking table
- Change descriptions

#### **3. Table of Contents**

- Section listing with page numbers

### **C.2 Introduction (Section 1)**

#### **1.1 Purpose**

- Document purpose
- Intended audience

#### **1.2 Scope**

- System boundaries
- Major features
-

### **1.3 Overview**

- Document organization

### **1.4 Reference Material**

- Related documents

### **1.5 Definitions and Acronyms**

- Terminology definitions

## **C.3 System Overview (Section 2)**

- High-level system description
- Context within larger system
- Major functions
- User characteristics

## **C.4 System Architecture (Section 3)**

### **3.1 Architectural Design**

- Overall structure
- Major components

### **3.2 Decomposition Description**

- Subsystem breakdown
- Component relationships

### **3.3 Design Rationale**

- Architecture justification
- Alternative considerations

## **C.5 Data Design (Section 4)**

- Data structures
- Database schema

## **4.2 Data Dictionary**

- Entity definitions
- Attribute descriptions

## **C.6 Component Design (Section 5)**

- Detailed component specifications
- Interface definitions
- Behavioral descriptions
- Pseudocode/algorithms

## **C.7 Human Interface Design (Section 6)**

### **6.1 Overview of User Interface**

- UI principles
- Navigation structure

### **6.2 Screen Images**

- UI mockups
- Screen layouts

### **6.3 Screen**

- Objects and Actions
- Interactive elements
- User actions

## **C.8 Requirements Matrix (Section 7)**

- Requirements traceability
- Functional requirements mapping
- Non-functional requirements mapping
- Verification methods

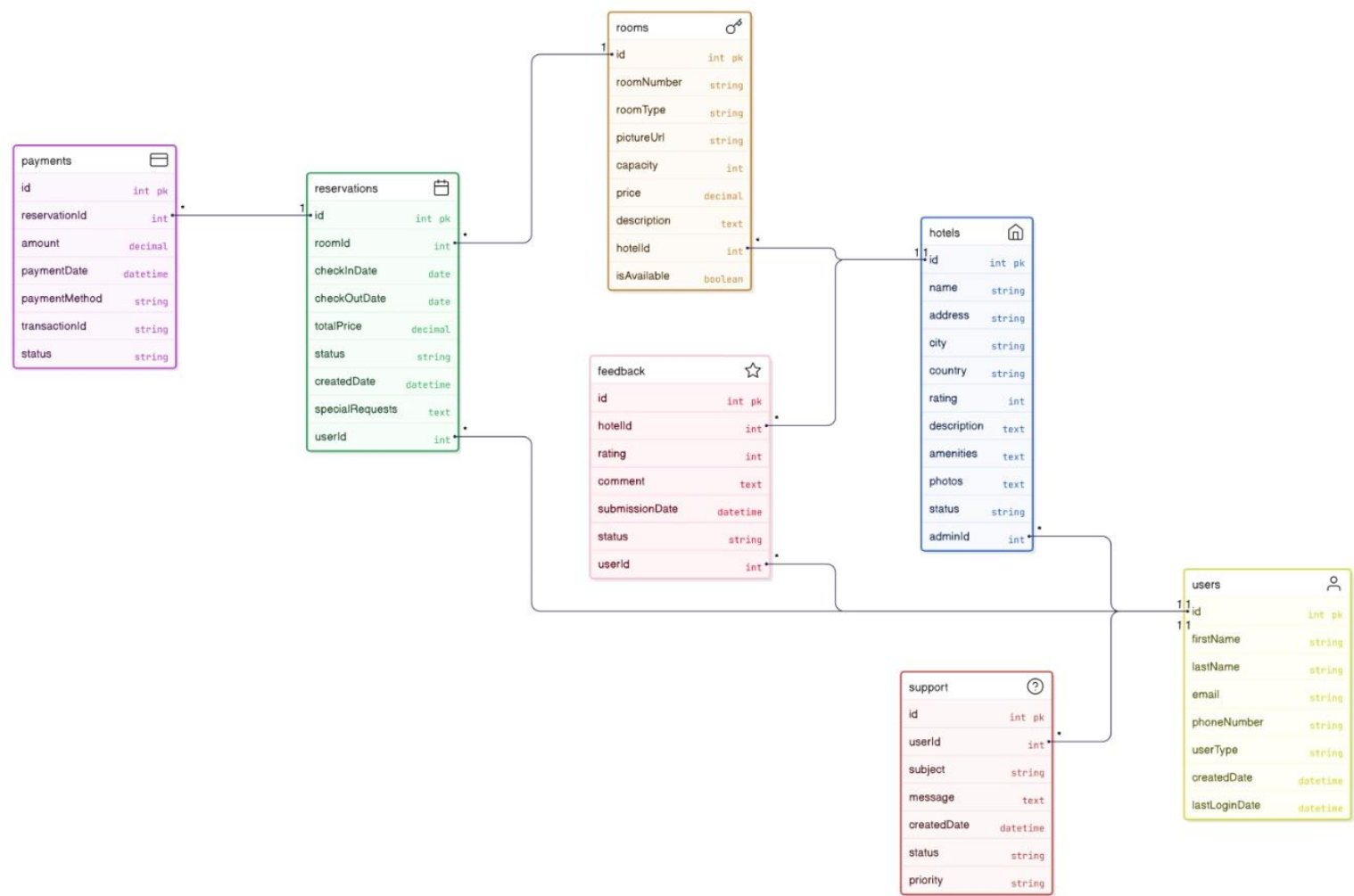
## **C.9 Appendices**

- Supporting documentation
- Technical specifications
- Additional diagrams
- Glossaries

Annex D: Supplementary Technical Documentation

D1) Database Schema diagram

Prestige Hotel Reservation System Data Model



D2) API Endpoints

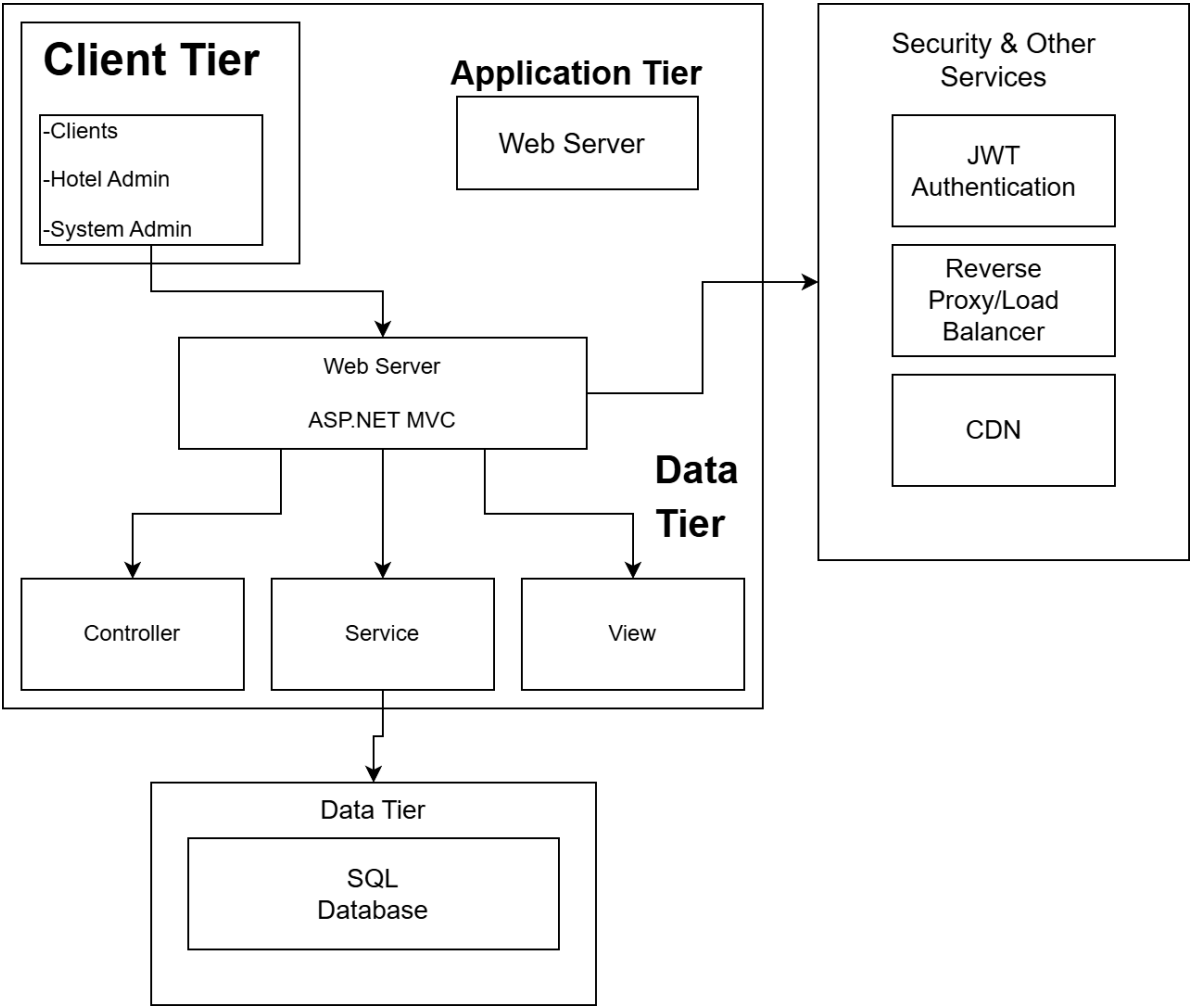
| Endpoint                     | Method | Description         | Authentication |
|------------------------------|--------|---------------------|----------------|
| /api/auth/login              | POST   | User login          | None           |
| /api/auth/register           | POST   | User registration   | None           |
| /api/auth/logout             | POST   | User logout         | Required       |
| /api/hotels                  | GET    | Get all hotels      | Optional       |
| /api/hotels/{id}             | GET    | Get hotel details   | Optional       |
| /api/hotels/search           | POST   | Search hotels       | Optional       |
| /api/bookings                | POST   | Create booking      | Required       |
| /api/bookings/{id}           | GET    | Get booking details | Required       |
| /api/bookings/user/{userId}  | GET    | Get user bookings   | Required       |
| /api/reviews                 | POST   | Submit review       | Required       |
| /api/reviews/hotel/{hotelId} | GET    | Get hotel reviews   | Optional       |
| /api/admin/hotels            | POST   | Add new hotel       | Admin only     |
| /api/admin/hotels/{id}       | PUT    | Update hotel        | Admin only     |
| /api/admin/users             | GET    | Get all users       | System Admin   |



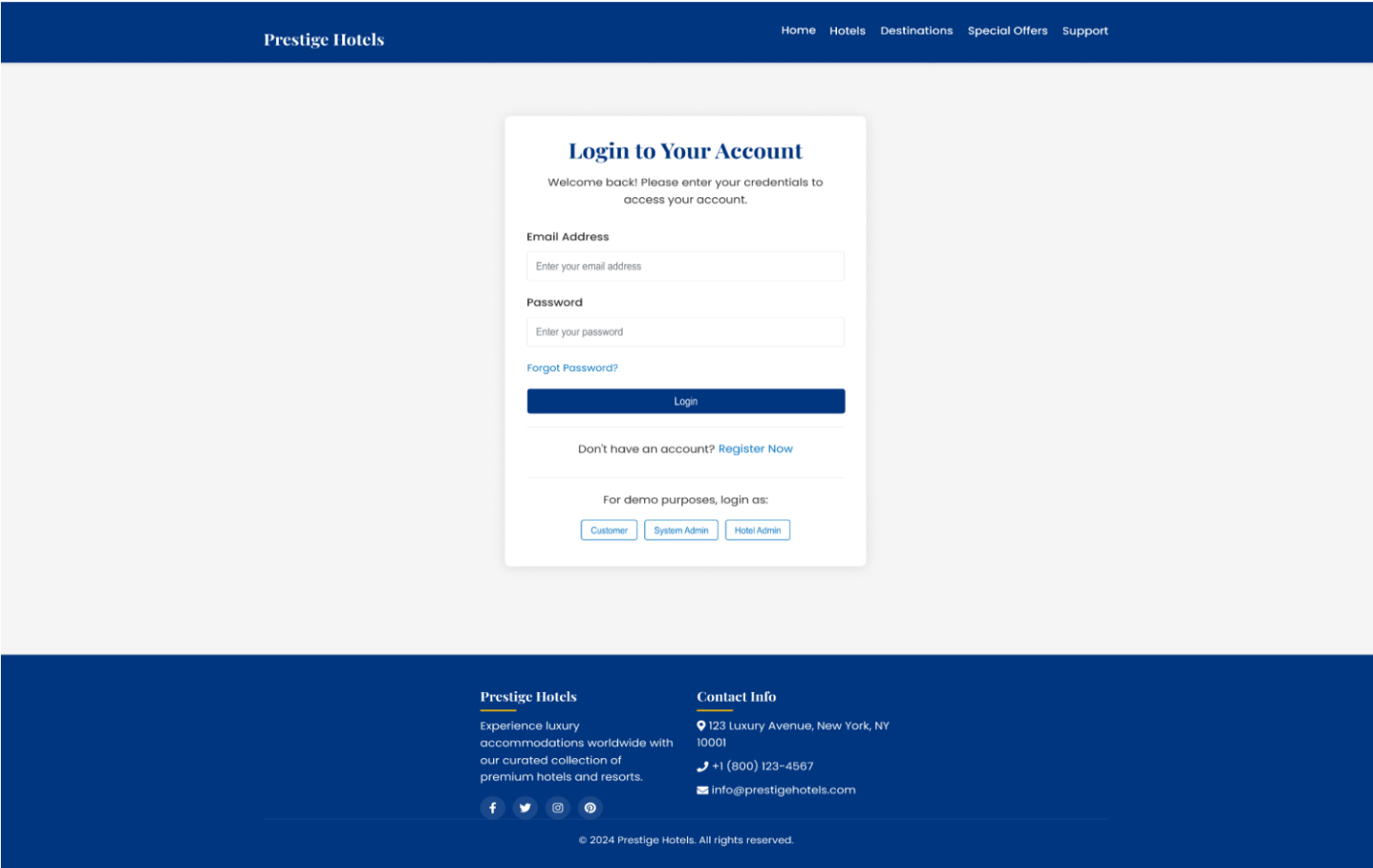
D3) Error Codes

| Code     | Description              | HTTP Status |
|----------|--------------------------|-------------|
| AUTH001  | Invalid credentials      | 401         |
| AUTH002  | Account locked           | 403         |
| AUTH003  | Token expired            | 401         |
| BOOK001  | Room not available       | 409         |
| BOOK002  | Payment failed           | 402         |
| BOOK003  | Invalid dates            | 400         |
| HOTEL001 | Hotel not found          | 404         |
| HOTEL002 | Invalid search criteria  | 400         |
| USER001  | User not found           | 404         |
| USER002  | Email already exists     | 409         |
| PERM001  | Insufficient permissions | 403         |

D4) Deployment Architecture



Annex E: Screen images Screen images



Login to Your Account

Welcome back! Please enter your credentials to access your account.

Email Address

Enter your email address

Password

Enter your password

[Forgot Password?](#)

Login

Don't have an account? [Register Now](#)

For demo purposes, login as:

Customer

System Admin

Hotel Admin

Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



Contact Info

123 Luxury Avenue, New York, NY 10001

+1 (800) 123-4567

info@prestigehotels.com

## Create an Account

Join Prestige Hotels to enjoy exclusive benefits and manage your bookings easily.

First Name

Last Name

Email Address

Phone Number

Password

Confirm Password

☐ I agree to the [Terms and Conditions](#) and [Privacy Policy](#)

☐ Subscribe to our newsletter to receive special offers and updates

Create Account

Already have an account? [Login](#)

### Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



### Contact Info

📍 123 Luxury Avenue, New York, NY 10001

📞 +1 (800) 123-4567

✉️ [info@prestigehotels.com](mailto:info@prestigehotels.com)

## Search Results

[Home](#) > [Search Results](#)

## Filters

Sort by:

- ☐ recommended
- ☐ min to max
- ☐ max to min

## Hotel Rating

- ☐ ★★★★★
- ☐ ★★★★☆
- ☐ ★★★☆☆

5 hotels found

Prestige Grand Hotel

Prestige Grand Hotel

€450

per night



Paris, France

★★★★☆ 4.8/5 (2 reviews)

Experience luxury in the heart of Paris with stunning views of the Eiffel Tower. Our 5-star accommodations offer the perfect blend of French elegance...

☐ Free WiFi ☐ Pool ☐ Spa ☐ Restaurant ☐ Fitness Center

[View Details](#)

Prestige Beachfront Resort

Prestige Beachfront Resort

€400

per night



Bali, Indonesia

★★★★☆ 4.8/5 (2 reviews)

Nestled on the pristine shores of Bali, our beachfront paradise offers private villas with infinity pools overlooking the crystal-clear waters of the...

☐ Private Beach ☐ Free WiFi ☐ Pool ☐ Spa ☐ Restaurant

[View Details](#)

Prestige Urban Boutique

Prestige Urban Boutique

€350

per night



New York, USA

★★★★☆ 4.5/5 (2 reviews)

Located in the vibrant heart of Manhattan, our boutique hotel combines contemporary design with unparalleled service, offering a sophisticated sanctuary...

☐ Free WiFi ☐ Restaurant ☐ Bar ☐ Fitness Center

☐ Business Center

[View Details](#)

Prestige Mountain Retreat

Prestige Mountain Retreat

€300

per night



Swiss Alps, Switzerland

★★★★☆ 4.9/5 (2 reviews)

Perched among the majestic Swiss Alps, our mountain retreat offers breathtaking panoramic views, ski-in/ski-out access, and cozy fireplaces in every room...

☐ Ski Access ☐ Free WiFi ☐ Spa ☐ Restaurant ☐ Bar

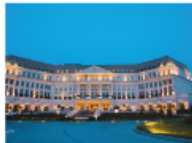
[View Details](#)

Prestige Historic Palace

Prestige Historic Palace

€380

per night



Venice, Italy

★★★★☆ 4.7/5 (2 reviews)

Housed in a beautifully restored 16th-century palazzo on the Grand Canal, our hotel offers a truly authentic Venetian experience with modern luxury amenities...

☐ Canal Views ☐ Free WiFi ☐ Restaurant ☐ Bar

☐ Concierge

[View Details](#)

## Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



## Contact Info

123 Luxury Avenue, New York, NY 10001

+1 (800) 123-4567

info@prestigehotels.com

search hotel by name?

## Experience Luxury, Create Memories

Discover our handpicked collection of the world's finest hotels and resorts

Where are you going?

10 . 05 . 2025



11 . 05 . 2025



2 Guests

Search Hotels

### Featured Hotels and Resorts

Discover our most popular properties worldwide

Prestige Grand Hotel

**Prestige Grand Hotel** €450  
per night

Paris, France

★★★★★ 4.8/5

Experience luxury in the heart of Paris with stunning views of the Eiffel Tower. Our 5-star accommod...

free WiFi AC room spa

[View Details](#)

Prestige Beachfront Resort

**Prestige Beachfront Resort** €400  
per night

Bali, Indonesia

★★★★★ 4.6/5

Nestled on the pristine shores of Bali, our beachfront paradise offers private villas with infinity ...

private beach free WiFi AC room

[View Details](#)

Prestige Mountain Retreat

**Prestige Mountain Retreat** €300  
per night

Swiss Alps, Switzerland

★★★★★ 4.9/5

Perched among the majestic Swiss Alps, our mountain retreat offers breathtaking panoramic views, ski...

ski access free WiFi spa

[View Details](#)

Prestige Villa

**Prestige Villa** €250  
per night

Venice, Italy

★★★★★

House centur...

car

11 Rooms

[View Details](#)

### Popular Destinations

Explore our most sought-after locations

Paris

**Paris, France**

Discover the city of lights with our curated selection of luxury accommodations.

5 Hotels

[Explore](#)

Bali

**Bali, Indonesia**

Experience paradise on earth with our exclusive beachfront villas and resorts.

3 Hotels

[Explore](#)

New York

**New York, USA**

Stay in the heart of the city that never sleeps with our premium urban retreats.

4 Hotels

[Explore](#)

#### Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



#### Contact Info

123 Luxury Avenue, New York, NY 10001

+1 (800) 123-4567

info@prestige-hotels.com

# Support And FAQ

[Home](#) > [Support](#)

## Frequently Asked Questions

### Bookings And Reservations

- How do I make a reservation? 
- Can I cancel or modify my reservation? 
- What payment methods are accepted? 

### Account Management

- How do I create an account? 
- I forgot my password. How do I reset it? 

### Hotels And Facilities

- What time is check-in and check-out? 
- Are pets allowed at your hotels? 
- Is breakfast included with my stay? 

## Contact Support

### Phone Support

 +1 800 123 4567  
Available 24/7

### Email Support

 support@prestigehotels.com  
We respond within 24 hours

### Send Us a Message

Your Name

Email Address

Subject

Message

Send Message

### Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



### Contact Info

 123 Luxury Avenue, New York, NY 10001  
 +1 (800) 123-4567  
 info@prestigehotels.com

Boran  
Bozkurt  
boranbozkurtoc@gmail.com



Dashboard

- My Reservations
- Favorite Hotels
- My Reviews
- Profile Settings
- Support
- Logout

Dashboard

Welcome back, John! Here's an overview of your account.

2

Active Reservations

2

Reviews

0

Saved Hotels

Upcoming Reservations

Booking #3

Confirmed

Prestige  
Grand Hotel

Prestige Grand Hotel

Paris, France

Junior Suite

Check-in

15 Mayıs 2024

Check-out

20 Mayıs 2024

Guests

2

Total Price

€3750

View Hotel

Manage Booking

Booking #5

Confirmed

Prestige  
Urban  
Boutique

Prestige Urban Boutique

New York, USA

City View Room

Check-in

10 Haziran 2024

Check-out

13 Haziran 2024

Guests

1

Total Price

€1050

View Hotel

Manage Booking

Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



Contact Info

123 Luxury Avenue, New York, NY 10001  
+1 (800) 123-4567  
info@prestigehotels.com

## Prestige Grand Hotel

★★★★★ 4.8/5

[Home](#) > [About](#) > [Prestige Grand Hotel](#)

Prestige Grand Hotel



Paris, France

### Overview

Experience luxury in the heart of Paris with stunning views of the Eiffel Tower. Our 5-star accommodations offer the perfect blend of French elegance and modern comfort.

### Amenities

|                |              |
|----------------|--------------|
| Free Wi-Fi     | Pool         |
| Spa            | Restaurant   |
| Fitness Center | Room Service |
| Concierge      |              |

### Our Rooms

Deluxe King Room



#### Deluxe King Room

2 Guests

Spacious room with king-sized bed, city views and luxury amenities.

✓ King Bed ✓ City View ✓ Air Conditioning ✓ Flat-screen TV ✓ Minibar ✓ Safe

€450

per night

[Book Now](#)

Junior Suite



#### Junior Suite

3 Guests

Elegant suite with separate living area and Eiffel Tower views.

✓ King Bed ✓ Suite Bed ✓ Eiffel Tower View ✓ Air Conditioning ✓ Flat-screen TV ✓ Minibar

€750

per night

[Book Now](#)

Presidential Suite



#### Presidential Suite

4 Guests

Luxurious suite with panoramic views, separate living and dining areas, and Butler service.

✓ King Bed ✓ Two Bedrooms ✓ Panoramic View ✓ Living Room ✓ Dining Area ✓ Butler Service

€1500

per night

[Book Now](#)

### Location

Hotel Location Map

#### Prestige Grand Hotel

123 Avenue des Champs-Élysées, 75008 Paris, France

#### Transportation

✈ Charles de Gaulle Airport - 20 km

🚇 Gare du Nord Station - 4.5 km

🚶 Champs-Élysées Metro Station - 0.3 km

### Guest Reviews

4.8

★★★★★

Based on 15 reviews

Cleanliness

4.8

Comfort

4.7

Location

4.9

Services

4.6

Value

4.5

John D.  
10 Jun 2023

★★★★★

Absolutely stunning hotel with impeccable service. The Eiffel Tower view from our room was breathtaking!

Sarah M.  
15 Jun 2023

★★★★☆

Great property in a perfect location. The staff was extremely helpful and friendly. Only downside was some noise from the street.

[See All Reviews](#)

#### Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



#### Contact Info

123 Luxury Avenue, New York, NY 10001

✆ +1 (212) 123-4567

✉ info@prestigehotels.com



Admin User

admin@example.com

Dashboard

Hotels

Users

Reservations

Reviews

Settings

Logout

System Dashboard

Welcome back, Admin! Here's an overview of the platform.



5

Hotels



3

Registered Users



3

Active Reservations



€42.850

Monthly Revenue

Recent Hotels

| ID | Hotel Name                 | Location        | Rating | Status | Actions              |
|----|----------------------------|-----------------|--------|--------|----------------------|
| #1 | Prestige Grand Hotel       | Paris, France   | 4.8    | Active | <a href="#">View</a> |
| #2 | Prestige Beachfront Resort | Bali, Indonesia | 4.6    | Active | <a href="#">View</a> |
| #3 | Prestige Urban Boutique    | New York, USA   | 4.5    | Active | <a href="#">View</a> |

Recent Users

| ID | Name          | Email                | Role        | Status | Actions              |
|----|---------------|----------------------|-------------|--------|----------------------|
| #1 | John Doe      | customer@example.com | Customer    | Active | <a href="#">View</a> |
| #2 | Admin User    | admin@example.com    | Admin       | Active | <a href="#">View</a> |
| #3 | Hotel Manager | hotel@example.com    | Hotel_admin | Active | <a href="#">View</a> |

Hotels Management

View and manage all hotels in the system

User Management

View and manage all users in the system

Reservations

View all reservations across the platform

Reviews

View and manage all reviews

manage complains

Manage global system settings

Hotel Manager

hotel@example.com



Dashboard

Hotel Details

Rooms

Reservations

Reviews

Profile Settings

Support

Logout

Hotel Dashboard

Welcome back, Hotel! Here's an overview of your hotel.

1  
Active Reservations

2  
Reviews

€4.250  
Monthly Revenue

Recent Reservations

| Booking ID | Guest    | Room         | Check In    | Check Out   | Status    | Actions                 |
|------------|----------|--------------|-------------|-------------|-----------|-------------------------|
| #3         | John Doe | Junior Suite | 15 May 2024 | 20 May 2024 | Confirmed | <a href="#">Details</a> |

Recent Reviews

Review

15 Eki 2023

Absolutely stunning hotel with impeccable service. The Eiffel Tower view from our room was breathtaking!

Review John Doe

Reply

★★★★★

Hotel Details

Manage your hotel information and settings

Room Management

Add, edit, and manage your hotel rooms, and price

Reservations

View and manage all hotel reservations

Reviews

View and respond to guest reviews

Profile Settings

Manage your account settings

Support

Get help with your hotel management

## Complete Your Booking

[Home](#) [Hotels](#) [Hotel Details](#) [Booking](#)

1

Room Selection

2

Guest Details

3

Payment

4

Confirmation

### Guest Information

#### Booking Details

Check In

88 - aa - yyyy



Check Out

88 - aa - yyyy



Guests

2 Guests

#### Personal Details

First Name

Last Name

Email Address

Phone Number

#### Special Requests

Special Requests (optional)

Let us know if you have any special requests...

#### Payment Details

Payment Method



Credit Card



PayPal

Card Number

XXXX XXXX XXXX XXXX

Name on Card

Expiry Date

MM/YY

CVV

XXX

☐ I agree to the [Terms and Conditions](#) and [Privacy Policy](#)[Complete Reservation](#)

### Booking Summary

#### Prestige Grand Hotel

Paris, France

★★★★☆

#### Deluxe King Room

2 Guests King Bed Free WiFi

Check-in

May 15, 2024

Check-out

May 20, 2024

Length of stay

5 Nights

Guests

2 Adults

Room Rate

€450 x 5 nights

Taxes &amp; Fees (10%)

€225

**Total****€2,475**

#### Cancellation Policy

Free cancellation until May 8, 2024. After this date, you will be charged the first night's rate.

#### Prestige Hotels

Experience luxury accommodations worldwide with our curated collection of premium hotels and resorts.



#### Contact Info

123 Luxury Avenue, New York, NY 10001

+1 (800) 123-4567

info@prestigehotels.com